

16-AI-REFACTORING-RULES.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Engineering Framework

Projet : Vitala

1. Objectif

AI-REFACTORING-RULES.md définit les règles obligatoires pour toute amélioration, restructuration ou optimisation du code Vitala.

Son objectif est de garantir que les améliorations :

- réduisent la complexité ;
 - améliorent la qualité ;
 - préservent les fonctionnalités existantes ;
 - ne créent pas de régression.
-

2. Définition du refactoring

Le refactoring est une modification interne du système visant à améliorer sa structure sans changer son comportement fonctionnel attendu.

Exemples :

- simplifier du code ;
- réduire la duplication ;
- améliorer l'organisation ;
- améliorer la lisibilité.

Le refactoring ne doit pas modifier le comportement utilisateur sans validation.

3. Principe fondamental

Une IA ne doit jamais refactoriser uniquement parce qu'elle préfère une autre approche.

Un refactoring doit résoudre un problème réel.

4. Raisons acceptables de refactoring

Un refactoring est justifié si :

- le code est difficile à maintenir ;
 - il existe une duplication importante ;
 - une dette technique bloque l'évolution ;
 - la sécurité est améliorée ;
 - les performances sont réellement problématiques ;
 - l'architecture est violée.
-

5. Raisons insuffisantes

L'IA ne doit pas refactoriser parce que :

- elle aurait écrit différemment ;
 - une nouvelle librairie existe ;
 - une nouvelle syntaxe est disponible ;
 - le code fonctionne déjà correctement.
-

6. Analyse obligatoire avant refactoring

Avant toute modification importante :

L'IA doit analyser :

```text id="b8q3mt" Code actuel

↓

Problème identifié

↓

Impact

↓

Solution proposée

↓

Risques

↓

#### Plan de migration

```

7. Principe de modification progressive

Les gros refactorings doivent être divisés.

Interdit :

Modifier 50 fichiers sans validation.

Préférable :

```text id="6k2p7s"
Petite modification

↓

Test

↓

Validation

↓

Étape suivante
```

8. Conservation du comportement

Un refactoring doit conserver :

- fonctionnalités ;
- API publiques ;
- contrats ;
- données ;
- permissions.

Toute modification comportementale doit être traitée comme une nouvelle fonctionnalité.

9. Refactoring frontend

L'IA peut améliorer :

- composants trop complexes ;
- duplication UI ;
- logique répétée ;
- performances de rendu.

Elle doit préserver :

- interface utilisateur ;
 - expérience utilisateur ;
 - accessibilité.
-

10. Refactoring backend

L'IA peut améliorer :

- organisation des services ;
- séparation des responsabilités ;
- validation ;
- gestion des erreurs.

Elle doit préserver :

- contrats API ;
 - logique métier ;
 - sécurité.
-

11. Refactoring Database

Les modifications Database sont critiques.

Avant tout changement :

Vérifier :

- migrations existantes ;
- dépendances ;
- API utilisant les données ;
- règles RLS.

Interdit :

- modifier une migration déjà exécutée ;

- supprimer une colonne sans stratégie ;
 - changer un schéma critique sans plan.
-

12. Refactoring architecture

Un changement architectural nécessite :

- analyse ;
- proposition ;
- validation ;
- documentation.

Une IA ne doit jamais restructurer tout Vitala automatiquement.

13. Refactoring de sécurité

Un refactoring améliorant la sécurité doit vérifier :

- permissions ;
- authentification ;
- accès données ;
- logs.

Une simplification qui réduit la sécurité est interdite.

14. Mesure d'amélioration

Après un refactoring, l'IA doit pouvoir expliquer :

Avant :

- problème ;
- limitation.

Après :

- amélioration obtenue.

Exemples :

- moins de duplication ;
 - meilleure lisibilité ;
 - temps réduit ;
 - meilleure sécurité.
-

15. Tests obligatoires après refactoring

Après toute modification significative :

L'IA doit vérifier :

[] Tests existants passent

[] Nouveaux tests ajoutés si nécessaire

[] Fonctionnalités principales vérifiées

[] Aucun comportement cassé

16. Gestion des risques

Pour chaque refactoring important :

Documenter :

``text id="x4r8ko" Objectif :

Fichiers concernés :

Risques :

Plan de retour arrière :

Résultat : ``

17. Refactoring interdit automatique

Une IA ne doit jamais lancer automatiquement :

✗ migration complète de framework.

✗ réécriture globale d'un module stable.

✗ changement d'architecture majeur.

✗ remplacement massif d'une technologie.

18. Dette technique

La dette technique doit être :

- identifiée ;
- documentée ;
- priorisée.

Elle ne doit pas être cachée.

19. Checklist avant validation

☐ Problème clairement identifié

☐ Refactoring justifié

☐ Impact analysé

☐ Tests validés

☐ Documentation mise à jour

☐ Aucun comportement cassé

20. Règle finale

Un bon refactoring rend Vitala plus simple.

Un mauvais refactoring rend Vitala différent mais pas meilleur.

L'objectif n'est pas de changer le code.

L'objectif est d'améliorer durablement le système.

Historique

Version	Modification
V1.0	Création des règles de refactoring IA

Fin de AI/16-AI-REFACTORING-RULES.md